

LServe + SampleAttention Slide Deck

Review online here, or download the prebuilt PDF for a LinkedIn document upload.

PDF: [./lservice-sampleattention.pdf](#)

Post URL: [../blog/2026-01-30-lservice-sampleattention-sparse-attention/](#)

ML INFRASTRUCTURE · PAPER BREAKDOWN

Sparse Attention for LLM Serving

TWO PAPERS, ONE IDEA

The win comes from structured sparsity that kernels can exploit cheaply.

Not arbitrary dropped scores. The shape of sparsity matters.

LServe

SERVING-SYSTEM VIEW

Use different sparse mechanisms for different bottlenecks.

PREFILL

 Skip blocks

DECODE

 Select pages

Gains are multiplicative: sparsity reduces page reads; quantization reduces each read.

Result: up to 2.9× prefill, 1.3–2.1× decode speedup vs vLLM on Llama-3-8B.

SampleAttention

SPARSE-STRUCTURE VIEW

Two patterns explain most of the useful attention mass during prefill.

COLUMN


Global anchors

SLASH


Local continuity

CRA — accuracy guardrail


Min retained attention mass per query

Adapts sparsity per head and prompt at runtime. No offline profile required.
Result: up to 2.42× TTFT vs FlashAttention.

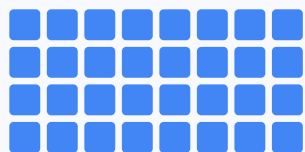
LSERVE · PREFILL PHASE

Block-Sparse + Streaming Heads

Goal: reduce sequential KV-block iterations when processing many query tokens at once.

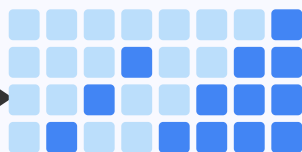
Block-Sparse Attention

DENSE (before LServe)



Every block computed — slow at long context

BLOCK-SPARSE (LServe)



■ Computed
■ Skipped

~60-70% fewer blocks — measurable speedup

Streaming Heads (Static Sparsity)

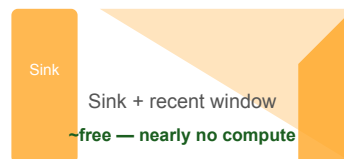
Dense Head



Full KV attention
 $O(n^2)$ cost

50% heads

Streaming Head (Λ -mask)



~free — nearly no compute

LSERVE · DECODE PHASE

Hierarchical KV-Page Selection

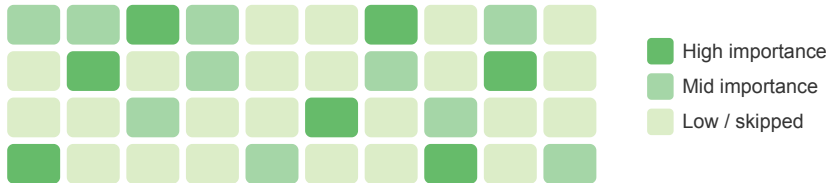
Goal: read fewer KV pages per decode step without missing important history.

The decode bottleneck is memory bandwidth, not compute

One token generated per step → attention over entire KV cache each step → GPU memory reads dominate
KV-cache can be 10s of GB. You read most of it every single decode step.

KV Cache as Physical Pages

Context = 40 KV pages stored in GPU memory



2-Level Hierarchical Selection

Level 1: Coarse (Constant-time)

Use a tiny summary vector per page.
Compare query against summaries.
 $O(n/\text{page_size})$ — cheap.



Top pages selected → sent to Level 2



Level 2: Fine (Token-level)

Within selected pages only:
run full attention per token.
Discard low-score tokens.



Only important tokens hit GPU HBM

Result: 1.3–2.1× decode throughput vs vLLM

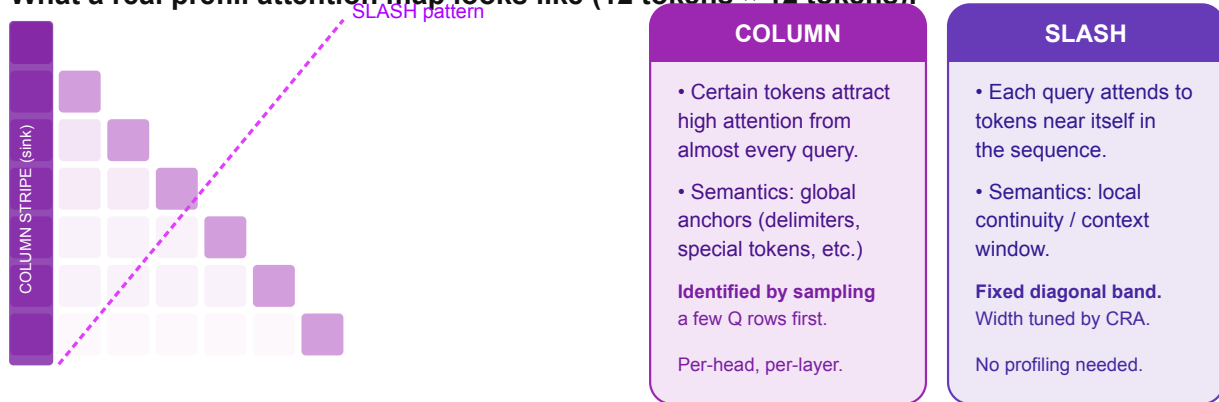
KV quantization (INT4 keys) stacks on top → combined bandwidth reduction of ~4-8×

SAMPLEATTENTION · SPARSE PATTERNS + CRA

Two Shapes in Every Attention Map

Real attention during prefill is not random — the same two patterns appear across heads and mode

What a real prefill attention map looks like (12 tokens × 12 tokens):



CRA — Cumulative Residual Attention

What it measures:

The fraction of total attention mass retained by the chosen sparse mask.

Sparse mask keeps this % of total attention:



Why it predicts accuracy: if you keep 90% of attention mass, the softmax output barely moves.
Approximated at runtime by sampling a few Q rows — no offline profiling, no fixed budget.

Mental Model: Three Takeaways

- ① Attention sparsity is structured. Two patterns win. Fine-grained sparsity doesn't scale to kernels.
- ② Use sparse mechanisms that match your bottleneck (block-skip for prefill, page-select for decode).
- ③ CRA gives you a quality signal for free — threshold it, don't fix token budgets statically.